Travel Order Resolver

Livrable du projet T-AIA-911

Réalisé par Romain CROCQ-MARCHAND, Justin MBODJE, Yasser MEKHFI, Mohamed FOFANA, Téophile CHARDON

Établissement de formation : EPITECH RENNES

Promotion 2026

1. Objectif du projet

Le but du projet est de prendre une phrase en français décrivant un trajet en train, et de retourner un itinéraire calculé automatiquement entre deux gares.

Exemple :

« Je veux aller de Rennes à Lyon »
→ détecter que c'est un trajet
→ extraire Rennes et Lyon
→ calculer le plus court chemin entre ces deux gares
→ retourner le chemin et la distance

Le point important n'était pas juste de faire fonctionner la solution, mais de construire un système permettant de **comparer différentes approches NLP proprement**, sans tout mélanger.

2. Problème technique réel

Il y a deux parties très différentes dans ce projet :

1. Comprendre une phrase en langage naturel (incertain, ambigu).
2. Calculer un plus court chemin dans un graphe (déterministe, formel).

Le vrai défi est la transition entre les deux.

Le NLP produit du texte ou des entités.
L'algorithme de graphe attend des identifiants précis (`station_id`).

Si le mapping est faux, tout le reste est faux.

Donc le cœur du problème, ce n'est pas juste "détecter une ville", c'est la transformer correctement en nœud du graphe.

3. Architecture choisie

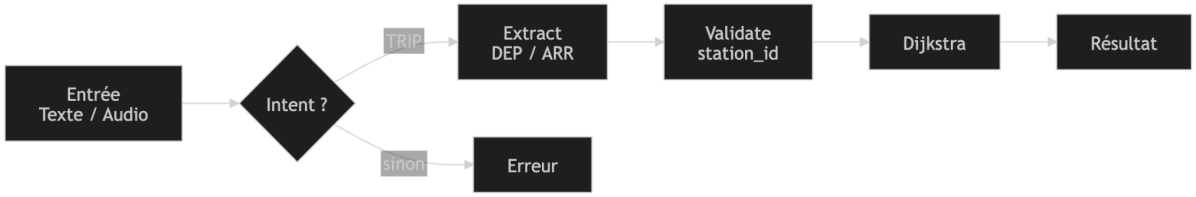
3.1 Vue d’ensemble de la pipeline

Nous avons structuré le système comme un pipeline clair :

1. Classification d’intention
2. Extraction des gares
3. Validation par rapport au référentiel
4. Chargement du graphe
5. Dijkstra
6. Retour du résultat
7. (Optionnel) visualisation sur carte

Chaque étape fait une seule chose.

Ce point était volontaire.
On voulait pouvoir changer le NLP sans toucher au graphe.
Et changer le graphe sans toucher au NLP.



Afin de clarifier le fonctionnement global du système, la figure illustre le pipeline complet.

Chaque étape transforme l’entrée en un format plus structuré, jusqu’à obtenir un résultat déterministe.

Ce découpage permet d’isoler les responsabilités et de comparer différentes approches NLP sans modifier les étapes suivantes.

3.1 Modules NLP interchangeables

Deux points de variation sont définis dans le pipeline :

Position	Rôle	Variantes disponibles
Étape 1	Classification d'intention	`RULE_BASED`, `FINETUNED_INTENT`, `HF_XNLI`
Étape 2	Extraction des gares (NER)	`RULE_BASED`, `HF_NER`, `SPACY`, `FINETUNED_NER`

Cela donne **12 combinaisons** évaluées systématiquement.

4. Données utilisées

4.1 Gares

Les gares sont stockées dans un fichier `stations.csv` :

Champ	Description
`station_id`	Clé interne (utilisée par tout le système)
`station_name`	Nom de la gare
`city`	Ville
`latitude` / `longitude`	Coordonnées géographiques

Le NLP ne retourne pas du texte libre, il retourne un `station_id`.

C'est ce qui permet d'enchaîner avec le graphe.

4.2 Graphe

Le réseau est stocké dans `edges.csv` :

Champ	Description
`from_station_id`	Gare de départ
`to_station_id`	Gare d'arrivée
`distance_km`	Distance en kilomètres

En mémoire, c'est une liste d'adjacence classique :

station -> [(voisin, distance)]

4.3 Dataset d'évaluation

10 000 exemples annotés dans `eval_10k.csv`, chaque ligne contenant :

- `texte` : la phrase brute
- `intention attendue` : TRIP / NOT_TRIP / NOT_FRENCH
- `departure_gt` : gare de départ (vérité terrain)
- `arrival_gt` : gare d'arrivée (vérité terrain)

Le dataset est **versionné dans le repo** et les évaluations sont automatisées via `pytest` avec génération de rapports PDF, garantissant la reproductibilité.

5. Algorithme de plus court chemin

Nous avons implémenté l'algorithme de Dijkstra afin de calculer le plus court chemin entre deux gares à partir du graphe construit depuis les fichiers CSV. L'algorithme a été codé manuellement afin de garantir une compréhension complète de son fonctionnement et de maîtriser son intégration dans le pipeline.

Dans notre cas, le graphe contient uniquement des distances positives, ce qui correspond parfaitement aux hypothèses classiques de Dijkstra. L'algorithme fournit un résultat déterministe et fiable, ce qui en fait un socle stable pour le reste du système.

La partie path finding n'est pas l'axe expérimental principal du projet. Elle sert de référence technique robuste afin d'évaluer correctement la qualité des stratégies NLP sans introduire d'incertitude supplémentaire au niveau du calcul de chemin.

En ouverture, une amélioration possible serait l'utilisation de l'algorithme A*. Contrairement à Dijkstra, A* exploite une heuristique — par exemple la distance géographique entre deux gares calculée à partir des coordonnées latitude/longitude — afin de guider la recherche et réduire le nombre de nœuds explorés. Sur un graphe de grande taille, cette approche permettrait d'accélérer significativement les temps de calcul tout en conservant l'optimalité du chemin, à condition que l'heuristique soit admissible.

6. Approches NLP implémentées

6.1 Rule-based

Approche simple :

> normalisation du texte

> recherche de noms de villes présents dans `stations.csv`

>prise des deux premières occurrences

C'est la baseline.

Elle est simple, rapide, et totalement déterministe.

Caractéristiques :

- Totalement déterministe
- Rapide (pas de modèle à charger)
- Fragile aux formulations nouvelles ou aux ambiguïtés (ex. *Lille* vs *île*, *Rennes* vs *reine*)

6.2 spaCy NER

On utilise spaCy pour détecter des entités de type lieu.

Ensuite on fait un mapping vers `station_id`.

Important :

spaCy ne "résout" pas la gare.

Il détecte juste un texte.

La résolution vers le référentiel est faite par nous.

Caractéristiques :

- Modèle généraliste non spécialisé sur les gares françaises
- Très rapide (P95 < 12 ms total)
- Performances faibles sur la tâche spécifique (Recall TRIP : ~18-24%)

6.3 HuggingFace NER

Même logique que spaCy, mais avec un modèle CamembERT.

Encore une fois :

Le modèle détecte une entité, mais le mapping vers la base de gares reste notre responsabilité.

Caractéristiques :

- Meilleure compréhension du français naturel et des formulations variées
- Meilleure généralisation que spaCy sur les entités géographiques
- Plus lent (P95 ~28-60 ms selon la combinaison)
- Taux d'erreur extracteur : ~2.99-9.53%

6.4 Fine-tuned NER

Principe : modèle NER fine-tuné spécifiquement sur notre dataset de gares.

Processus d'entraînement :

- Données d'entraînement : phrases annotées avec entités DEP/ARR correspondant aux noms de gares du référentiel
- Fine-tuning à partir d'un modèle de base CamemBERT
- Évaluation sur jeu de test séparé

Caractéristiques :

- Meilleur recall départ (88.35%) vs HF généraliste (89.81%) — légèrement inférieur
- Taux d'erreur extracteur : ~4.15-12.26%

6.5 Fine-tuned Intent Classifier

Principe : classificateur d'intention entraîné sur nos 3 classes : TRIP / NOT_TRIP / NOT_FRENCH.

Processus d'entraînement :

- Données : dataset annoté avec les 3 intentions
- Fine-tuning d'un modèle de classification de séquence
- Paramètres finaux : Intent Accuracy = **87.00%** (vs 84.4-84.6% pour rule-based)

Caractéristiques :

- Meilleur recall TRIP (87.48-87.51%) au prix d'une précision légèrement inférieure (77.63-88.54%)
- Latence élevée (~22-24 ms pour la classification seule)
- Filtre par intention plus permissif : seulement 49% des phrases filtrées (vs 69% pour rule-based)

6.6 Fine-tuned Intent Classifier

Phrase d'entrée : `"je veu aller de rene a lion"`

Étape 1 — Classification d'intention :

- Rule-based : normalisation → détecte "aller de ... à" → `TRIP` ✓
- Fine-tuned : tokenisation → score TRIP = 0.94 → `TRIP` ✓

Étape 2 — Extraction des entités :

- Rule-based : `rene` → fuzzy match → `Rennes` | `lion` → fuzzy match → `Lyon` ✓
- HF NER : `B-LOC: "rene"`, `B-LOC: "lion"` → résolution → `Rennes`, `Lyon` ✓
- spaCy : entités non reconnues (fautes d'orthographe) → échec ✗

Étape 3 — Résolution vers station_id :

- `"Rennes"` → `station_id: 87_471003`
- `"Lyon"` → `station_id: 87_722025`

Étape 4 — Dijkstra :

- Entrée : `(87_471003, 87_722025)`
- Chemin trouvé : Rennes → Le Mans → Tours → Massy → Lyon (~10 stations)
- Distance : ~614 km

Sortie finale :

```
{
  "departure": "Rennes",
  "arrival": "Lyon",
  "path": ["Rennes", "Le Mans", "Tours", "Massy TGV", "Lyon Part-Dieu"],
  "distance_km": 614.2
}
```

7. Résultats et comparaison des stratégies

7.1 Tableau de synthèse complet

Métrique	RB→RB	RB→HF	RB→SPACY	RB→FT_NER	FI→RB	FI→HF	FI→SPACY	FI→FT_NER
Accuracy globale	86.59%	86.18%	66.12%	84.97%	84.30%	**89.82%**	66.05%	88.18%
Precision (TRIP)	100%	100%	100%	100%	77.63%	87.97%	80.36%	88.54%
Recall (TRIP)	67.78%	66.79%	18.60%	63.89%	87.48%	**87.51%**	24.39%	82.24%
F1 Score (TRIP)	80.80%	80.09%	31.36%	77.97%	82.26%	**87.74%**	37.42%	85.28%
Intent Accuracy	84.64%	84.44%	84.42%	84.39%	**87.00%**	**87.00%**	**87.00%**	**87.00%**
Départ correct	73.16%	68.77%	39.50%	68.74%	**94.59%**	89.81%	52.09%	88.35%
Arrivée correcte	67.83%	66.79%	18.60%	63.89%	87.53%	**87.51%**	24.39%	82.24%
Both correct	67.78%	66.79%	18.60%	63.89%	87.48%	**87.51%**	24.39%	82.24%
Paths found	100%	100%	100%	100%	100%	100%	100%	100%
Latence moyenne	10.72 ms	14.56 ms	5.99 ms	13.19 ms	32.86 ms	35.54 ms	27.40 ms	37.64 ms
Latence P95	25.00 ms	37.18 ms	**12.04 ms**	34.42 ms	50.14 ms	56.75 ms	36.99 ms	61.14 ms

7.2 Analyse par axe

Qualité d'extraction (Both correct)

Les combinaisons avec **FINETUNED_INTENT** en classifier surpassent nettement le rule-based sur l'extraction :

- FI→RB : **87.48%** — la meilleure extraction de départ (94.59%) compense un NER simple
- FI→HF : **87.51%** — meilleur équilibre global
- RB→RB : 67.78% — limité par le recall du classifier rule-based

spaCy est clairement inadapté à cette tâche : 18.60% (RB→SPACY) et 24.39% (FI→SPACY).

Précision vs Rappel

Les combinaisons **RB→*** ont une précision TRIP de 100% mais un recall faible (~18-68%) : elles sont très conservatrices, ne classifiant TRIP que les cas évidents.

Les combinaisons **FI→*** sacrifient un peu de précision (77-88%) pour un recall bien supérieur (82-87%) — meilleur F1 global.

Performance / Latence

Profil	**Meilleure combinaison**	**Cas d'usage**
Précision maximale	**FI→HF** (F1 87.74%)	Production, qualité prioritaire
Vitesse maximale	**RB→SPACY** (5.99 ms)	Temps réel, mais qualité médiocre
Compromis	**FI→FT_NER** (F1 85.28%, 37 ms)	Si HF indisponible

Profil	Meilleure combinaison	Cas d'usage
Baseline solide	**RB→RB** (F1 80.80%, 10 ms)	Simplicité, pas de dépendance modèle

7.3 Impact du fine-tuning

Composant	Avant (base)	Après fine-tuning	Gain
Classifier d'intention	84.4-84.6% (rule-based)	**87.0%**	+~2.5 pts
NER - Both correct (avec FI)	66.79% (HF généraliste)	82.24% (FT_NER)	+15.5 pts vs spaCy base
F1 Score global	80.78% (RB→RB baseline)	**87.74%** (FI→HF)	+7 pts

Le fine-tuning de l'intent classifier apporte un gain homogène sur toutes les combinaisons (~+2.5 pts d'Intent Accuracy). Le gain principal vient du **meilleur recall** (87% vs 67%) au prix d'une légère baisse de précision.

8. Métriques utilisées

8.1 Exact-match

Un cas est correct si :

- l'intention est correcte
- la gare de départ est correcte
- la gare d'arrivée est correcte

C'est strict.

Si une seule gare est fausse → échec.

C'est volontairement exigeant.

8.2 Precision / Recall / F1

On calcule aussi :

- **Précision** : parmi les phrases classifiées TRIP, combien le sont vraiment ?
- **Rappel** : parmi les vraies phrases TRIP, combien sont détectées ?
- **F1** : moyenne harmonique — mesure l'équilibre entre les deux

Ça permet de voir si une stratégie :

- détecte trop de trajets
- ou en rate trop

8.3 Métriques temporelles

Mesure de la moyenne, médiane et P95 sur 10 000 appels. Attention au **cold start** des modèles NLP (premiers appels biaisés). Le pathfinding Dijkstra contribue moins de 1.25 ms dans tous les cas.

9. Ce qu’on a réellement mesuré

Important :

On mesure surtout :

- la qualité d’extraction
- la capacité à produire une route valide

On ne mesure pas :

- si le chemin est réellement optimal par rapport à une vérité terrain externe
- si la distance correspond à une vraie distance SNCF

Le graphe est fonctionnel, mais pas benchmarké contre des données officielles.

10. Limites identifiées

10.1 NLP

- Rule-based fragile aux formulations nouvelles.
- NER dépend fortement du mapping vers `station_id`.
- Ambiguïtés possibles (Lille vs île, Rennes vs reine, etc.).

10.2 Données

- Dataset en partie synthétique.
- Pas d’annotation du chemin attendu.
- Exact-match très strict.

10.3 Expérimental

- Toutes les stratégies ne sont pas évaluées exactement de la même manière.
- Effets de cache possibles sur les temps.

11. Conclusion

Le Travel Order Resolver est un système modulaire permettant de comparer différentes stratégies NLP dans un cadre contrôlé et reproductible. La séparation claire entre la compréhension du langage naturel et le calcul de chemin permet d'évaluer les modèles sans perturber le reste du pipeline. Le graphe et Dijkstra servent de base stable et déterministe.

Le système fonctionne de bout en bout, de la phrase brute jusqu'à l'itinéraire calculé. L'évaluation automatisée sur un dataset versionné permet une comparaison objective en termes de précision, rappel, F1 et latence.

Cependant, plusieurs limites subsistent. La résolution des entités vers les `station_id` reste un point sensible, notamment en cas d'ambiguïtés ou de bruit. De plus, les métriques utilisées mesurent la cohérence interne du pipeline, mais pas la validité réelle du réseau ferroviaire.

Les principales difficultés ont concerné la transition entre texte libre et représentation structurée du graphe. En perspective, le projet pourrait être amélioré par un dataset plus complet, une meilleure résolution d'entités ou l'utilisation d'algorithmes comme A* pour optimiser le calcul de chemin.

Ce projet constitue une base solide pour analyser et comparer des approches NLP dans un cadre expérimental maîtrisé.